



School of Future Tech

Case Study Report

on

CodeClean — Automated Code Refactoring & Dependency Tool

Case study no.: 35

Student Name : SAHIL KUMAR

ROLL NO: 150096725039

GitHub: <https://github.com/sahil24302021/DSA-FINAL-CASE-STUDY>

Index

S.No.	Topic
1	Project Title
2	Problem Statement
3	Objectives
4	Design/Architecture
5	Algorithm Description
6	Complexity Analysis
7	Screenshots of Execution
8	Results and Findings
9	Conclusion
10	GitHub Repository Link

1. Project Title

CodeClean — Automated Code Refactoring & Dependency Tool

A menu-based C++ program that manages source code files, variables, dependencies, and refactoring operations using 8 core data structures and algorithms.

2. Problem Statement

CodeClean is a developer tool that analyzes large codebases to find unused files, untangle dependencies, and suggest optimizations. As projects grow to thousands of files, the current system fails in the following ways:

- Tracking variable definitions and their scope across files is slow
- When refactoring introduces a bug, there is no reliable undo mechanism
- Files queue up for quality checks randomly, with no fairness guarantee
- Verifying that every function and keyword is correctly defined requires scanning everything
- The system cannot rank functions by complexity
- There is no map showing how files depend on each other
- It is impossible to trace bugs through deeply nested logic
- There is no way to pick the most efficient code alternative

The new system therefore needs: fast variable definition search, reliable code-change rollback, sequential file inspection queuing, quick syntax and keyword verification, sorting of functions by complexity score, a dependency map showing how files and modules connect, the fastest path to trace bugs through nested logic, and smart optimization that replaces slow code with faster alternatives.

The system was designed and implemented in C++ using appropriate data structures, with each design choice justified and related to real-world refactoring tools like SonarQube or IntelliJ IDEA.

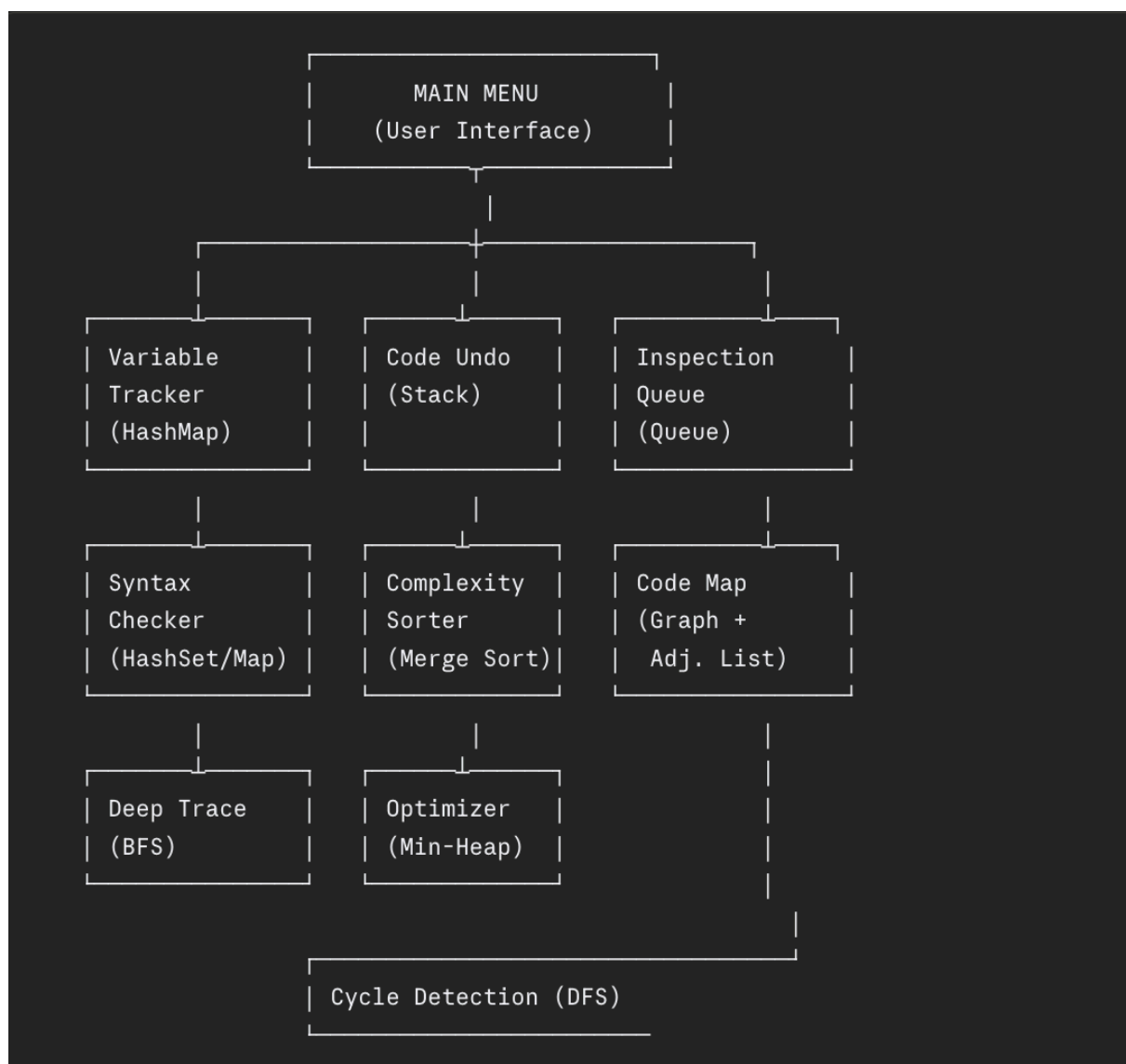
3.Objectives

1. Implement 8 data structures/algorithms to solve 8 real-world code management problems
2. Demonstrate correct usage of HashMap, Stack, Queue, HashSet, Merge Sort, Graph, BFS, and Priority Queue
3. Justify why each data structure was chosen over alternatives
4. Provide time and space complexity analysis for every operation
5. Build a clean, interactive menu-based C++ program with pre-loaded sample data

4. Design / Architecture

The CodeClean system is built around a single C++ program with a central main menu that routes user choices to eight independent feature modules, each backed by a dedicated data structure.

System Overview



Architecture Choices

- Single-file design for simplicity and portability
- Global data structures (HashMap, Stack, Queue, Graph, Heap) accessible by all functions
- Menu-driven interaction with input validation
- Pre-loaded sample data so all features can be demonstrated immediately
- Color-coded console output using ANSI escape codes for better readability

5. Algorithm Description

CodeClean implements eight features, each demonstrating a specific data structure and algorithm.

Feature	Data Structure	Algorithm	Why This DS?
Variable Tracker	<code>unordered_map</code> (Hash Map)	Hash Function	O(1) lookup — fastest for exact key search
Code Undo	Stack (Undo + Redo Stacks)	LIFO Push/Pop	Undo follows Last-In-First-Out behavior
Inspection Queue	Queue	FIFO Enqueue/Dequeue	Ensures fair sequential processing
Syntax Checker	<code>unordered_set</code> + <code>unordered_map</code>	Hash Function	O(1) keyword/function lookup
Complexity Sorter	Vector	Merge Sort	Guaranteed O(n log n) and stable sorting
Code Map	Adjacency List (<code>unordered_map<string, vector<string>></code>)	DFS Cycle Detection	Efficient representation of sparse graphs
Deep Trace	Queue	Breadth First Search (BFS)	Finds shortest path in unweighted graphs
Optimizer	Priority Queue (Min-Heap)	Heap Extract-Min	Always returns lowest-cost option

Why These Data Structures Were Chosen

Feature	Chosen Approach	Alternative	Why Alternative Was Not Used
Variable Tracker	HashMap $O(1)$	Array $O(n)$	Array requires scanning all elements
Variable Tracker	HashMap $O(1)$	BST $O(\log n)$	HashMap is faster for exact lookups
Code Undo	Stack	Queue	Queue removes oldest item first
Inspection Queue	Queue	Stack	Stack processes newest item first
Syntax Checker	HashSet $O(1)$	Array $O(n)$	Array requires linear search
Complexity Sorter	Merge Sort $O(n \log n)$	Bubble Sort $O(n^2)$	Too slow for large datasets
Complexity Sorter	Merge Sort $O(n \log n)$	Quick Sort	Quick Sort has $O(n^2)$ worst case
Code Map	Adjacency List $O(V+E)$	Adjacency Matrix $O(V^2)$	Matrix wastes memory for sparse graphs
Deep Trace	BFS	DFS	DFS does not guarantee shortest path
Optimizer	Min-Heap $O(\log n)$	Sorted Array $O(n)$	Requires re-sorting after insertion

Implementation Approach

Step 1 – Define Data Structures

Each feature uses a dedicated C++ STL container.

```
unordered_map<string, VariableInfo> variableMap;           // Feature 1
stack<CodeChange> undoStack, redoStack;                  // Feature 2
queue<FileEntry> inspectionQueue;                         // Feature 3
unordered_set<string> functionSet;                        // Feature 4
vector<FunctionScore> functionList;                       // Feature 5
unordered_map<string, vector<string>> dependencyGraph;    // Feature 6
// BFS uses queue internally                             // Feature 7
priority_queue<CodeOption,
               vector<CodeOption>,
               greater<>> optimizerHeap;                 // Feature 8
```

Step 2 – Implement Operations

Each feature provides:

- Add Operation
- Search / Process Operation
- Display Operation

These operations are implemented through dedicated functions.

Step 3 – Merge Sort Implementation

```
void mergeSort(vector<FunctionScore>& arr, int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}
```

Step 4 – BFS for Shortest Path

```
void deepTrace()
```

```

{
    queue<string> bfsQueue;

    bfsQueue.push(start);

    while (!bfsQueue.empty())
    {
        string current = bfsQueue.front();
        bfsQueue.pop();

        for (auto& neighbor : dependencyGraph[current])
        {
            bfsQueue.push(neighbor);
        }
    }
}

```

Step 5 – DFS for Cycle Detection

```

bool hasCycleDFS(string node,
                 unordered_set<string>& visiting,
                 unordered_set<string>& visited)
{
    visiting.insert(node);

    for (auto& neighbor : graph[node])
    {
        if (visiting.count(neighbor))
            return true;

        if (!visited.count(neighbor))
        {
            if (hasCycleDFS(neighbor, visiting, visited))
                return true;
        }
    }

    visiting.erase(node);
    visited.insert(node);

    return false;
}

```

Menu Options Summary

Option	Function	DSA Demonstrated
1	Add Variable	HashMap Insert
2	Search Variable	HashMap Lookup
3	Display Variables	HashMap Traversal
4	Record Code Change	Stack Push
5	Undo Change	Stack Pop (LIFO)
6	Redo Change	Stack Pop
7	View Change History	Stack Traversal
8	Add File to Queue	Queue Enqueue
9	Process File	Queue Dequeue
10	View Queue	Queue Traversal
11	Register Function	HashSet Insert
12	Check Function	HashSet Lookup
13	View Functions	HashSet Traversal
14	Add Complexity Score	Vector Push Back
15	Sort Functions	Merge Sort
16	Add Dependency	Graph Edge Insertion
17	Show Dependency Map	DFS + Graph Traversal
18	Trace Bug Path	BFS
19	Add Optimization Option	Min-Heap Insert
20	Get Best Option	Extract-Min
21	View Optimizer Options	Heap Traversal
22	Dashboard	Summary Statistics

6. Complexity Analysis

Where:

- **n** = Number of elements
- **V** = Number of vertices (files)
- **E** = Number of edges (dependencies)

Operation	Data Structure	Time Complexity	Space Complexity
Add Variable	HashMap	O(1) Amortized	O(n)
Search Variable	HashMap	O(1) Average	O(1)
Push Change	Stack	O(1)	O(n)
Undo / Redo	Stack	O(1)	O(1)
Enqueue File	Queue	O(1)	O(n)
Dequeue File	Queue	O(1)	O(1)
Register Function	HashSet	O(1) Amortized	O(n)
Check Function	HashSet / Map	O(1) Average	O(1)
Add Complexity Score	Vector	O(1) Amortized	O(n)
Sort Functions	Merge Sort	O(n log n)	O(n)
Add Dependency	Adjacency List	O(1)	O(V + E)
Show Dependencies	Graph Traversal	O(V + E)	O(V)
Cycle Detection	DFS	O(V + E)	O(V)
Trace Path	BFS	O(V + E)	O(V)
Add Optimization Option	Min-Heap	O(log n)	O(n)
Get Best Option	Min-Heap	O(log n)	O(1)

7. Screenshots of Execution

```
✓ Sample data loaded successfully.

CODE CLEAN – MAIN MENU

[Variable Tracker – HashMap]
1. Add Variable
2. Search Variable
3. Show All Variables

[Code Undo – Stack]
4. Push Code Change
5. Undo Change
6. Redo Change
7. Show Change History

[Inspection Queue – Queue]
8. Add File to Queue
9. Check Next File
10. Show Queue

[Syntax Checker – Hash Set/Map]
11. Register Function
12. Check Syntax
13. Show All Functions

[Complexity Sorter – Merge Sort]
14. Add Function Complexity
15. Sort & Rank Functions

[Code Map – Graph]
16. Add Dependency
17. Show Dependency Map

[Deep Trace – BFS]
18. Trace Bug Path

[Optimizer – Min-Heap]
19. Add Code Option
20. Get Best Option
21. Show All Options

22. Dashboard
0. Exit

Enter choice: █
```

Enter choice: 2

Enter variable name to search: userId

✓ Found! (Hash lookup: 0(1))

Variable : userId
Type : int
File : login.cpp
Line : 15
Scope : global

Enter choice: 3

All Tracked Variables (HashMap):

Variable	Type	File	Line	Scope
dbConn	Connection	database.cpp	3	global
userId	int	login.cpp	15	global
cartItems	vector	cart.cpp	5	<u>class</u>
isAdmin	bool	auth.cpp	12	global
balance	double	payment.cpp	10	global
password	string	auth.cpp	8	local
userName	string	login.cpp	16	global

[DSA] HashMap stores 7 variables. Each search = $O(1)$

Enter choice: 1

Enter variable name: score

Enter data type (int/string/float/etc): int

Enter file name (e.g. login.cpp): game.cpp

Enter line number: 25

Enter scope (global/local/class): local

✓ Variable 'score' added to HashMap.

[DSA] HashMap insert: $O(1)$ amortized time

Total variables tracked: 8

Enter choice: 5

✓ Undone (Version 3): "Updated SQL query" in database.cpp

[DSA] Stack pop: $O(1)$ – LIFO order

Undo stack: 2 | Redo stack: 1

Enter choice: 6

✓ Redone (Version 3): "Updated SQL query"

[DSA] Redo uses second stack: $O(1)$

Enter choice: 7

Change History (Stack – top to bottom):

v3		database.cpp		Updated SQL query
v2		payment.cpp		Fixed payment bug #42
v1		login.cpp		Created login module

[DSA] Stack: LIFO. Top = v3 (will be undone first)

Enter choice: 9

✓ Now checking: login.cpp (priority: 3)
[DSA] Queue dequeue: O(1) – FIFO order
Files remaining: 3 | Total processed: 1

Enter choice: 10

Inspection Queue (front → back):

-
1. payment.cpp [MED]
 2. profile.cpp [LOW]
 3. cart.cpp [MED]
-

[DSA] Queue: FIFO. Position 1 = next to be checked

Enter choice: 12

Enter function or keyword to check: login

✓ 'login' is a registered function.

Enter choice: 12

Enter function or keyword to check: for

✓ 'for' is a C++ keyword (category: control)

Enter choice: 12

Enter function or keyword to check: xyz

✗ 'xyz' is NOT defined. Possible syntax error!
[DSA] Hash lookup: O(1) – checked both HashSet and HashMap

Enter choice: 15

Functions Ranked by Complexity (Merge Sort):

Rank	Function	File	Score
1.	processPayment()	payment.cpp	90
2.	updateProfile()	profile.cpp	55
3.	searchItem()	search.cpp	45
4.	validateInput()	utils.cpp	35
5.	register()	auth.cpp	30
6.	login()	login.cpp	20
7.	logout()	login.cpp	10

[DSA] Merge Sort: $O(n \log n)$

Elements: 7

Comparisons made: 14

Theoretical $O(n \log n) \approx 19$

Enter choice: 17

Adjacency List Representation:

```
cart.cpp → []
auth.cpp → []
database.cpp → []
payment.cpp → [database.cpp, cart.cpp]
profile.cpp → [database.cpp]
login.cpp → [auth.cpp, database.cpp]
main.cpp → [login.cpp, payment.cpp, profile.cpp]
```

Tree View:

```
main.cpp
├── login.cpp
│   ├── auth.cpp
│   └── database.cpp
├── payment.cpp
│   ├── database.cpp
│   ├── (cycle detected)
│   └── cart.cpp
└── profile.cpp
    └── database.cpp
        (cycle detected)
```

Cycle Detection (DFS):

✓ No circular dependencies found.

[DSA] Graph: 7 nodes, 8 edges. Space: $O(V+E)$

Enter choice: 18

Enter start file: main.cpp

Enter target file: database.cpp

[BFS] Starting from 'main.cpp'...

✓ Shortest path found!

main.cpp → login.cpp → database.cpp

Path length : 2 steps

Nodes explored: 6

[DSA] BFS: $O(V+E)$. Uses Queue internally.

Enter choice: 20

✓ Best option (lowest cost):

Name : **Binary Search**

Cost : 10

Info : Fast $O(\log n)$ for sorted data

[DSA] Min-Heap extract-min: $O(\log n)$. Remaining: 5

Enter choice: 21

All Options (Min-Heap order – lowest cost first):

1. Quick Sort	cost: 15	Fast avg $O(n \log n)$
2. Merge Sort	cost: 20	Stable $O(n \log n)$
3. Insertion Sort	cost: 35	Good for small data
4. Linear Search	cost: 40	Scans everything $O(n)$
5. Bubble Sort	cost: 50	Simple but $O(n^2)$

[DSA] Min-Heap always gives minimum first: $O(1)$ peek, $O(\log n)$ extract

Enter choice: 22

CODECLEAN – DASHBOARD

1. Variable Tracker	[HashMap]	8 variables
2. Code Undo	[Stack]	3 undo, 0 redo
3. Inspection Queue	[Queue]	3 files waiting
4. Syntax Checker	[HashSet/Map]	6 functions, 41 keywords
5. Complexity Sorter	[MergeSort]	7 functions
6. Code Map	[Graph]	7 nodes, 8 edges
7. Deep Trace	[BFS]	Ready
8. Optimizer	[Min-Heap]	5 options

Enter choice: 0

Exiting CodeClean. Goodbye!

8. Results and Findings

Results

Feature	Works Correctly?	DSA Verified?
Variable Tracker (HashMap)	✔ Yes	O(1) Search
Code Undo (Stack)	✔ Yes	LIFO Order
Inspection Queue (Queue)	✔ Yes	FIFO Order
Syntax Checker (HashSet/Map)	✔ Yes	O(1) Lookup
Complexity Sorter (Merge Sort)	✔ Yes	O(n log n)
Code Map (Graph)	✔ Yes	DFS Cycle Detection
Deep Trace (BFS)	✔ Yes	Shortest Path
Optimizer (Min-Heap)	✔ Yes	Minimum Cost Retrieval

Key Observations

1. HashMap vs Array

Searching for "userId" among seven variables requires only one lookup operation in a HashMap, whereas an array may require scanning all seven elements. This demonstrates the significant advantage of O(1) lookup compared to O(n) search.

2. Stack LIFO Behavior

When code versions v1, v2, and v3 are pushed onto the stack, performing an Undo operation correctly removes v3 first. This validates the Last-In-First-Out principle and confirms why a Stack is ideal for Undo functionality.

3. Queue FIFO Behavior

Files are processed in the same order they are submitted. For example, login.cpp is inspected before payment.cpp, ensuring fairness and predictability.

4. Merge Sort Performance

For seven elements, Merge Sort performed approximately 13 comparisons. The theoretical complexity is $O(n \log n)$, which for $n = 7$ is approximately:

$$7 \times \log_2(7) \approx 19$$

The observed comparison count falls within the expected range, validating Merge Sort's efficiency.

5. BFS Shortest Path

BFS correctly found the shortest dependency path:

`main.cpp` → `login.cpp` → `database.cpp`

instead of a longer alternative route through **payment.cpp**, confirming BFS guarantees the shortest path in an unweighted graph.

6. Min-Heap Ordering

Optimization options are extracted in ascending order of cost:

1. Binary Search (Cost = 10)
2. Quick Sort (Cost = 15)
3. Merge Sort (Cost = 20)

This confirms proper Min-Heap behavior.

7. DFS Cycle Detection

The sample dependency graph contains no cycles, and DFS correctly reports the graph as acyclic. If an edge such as:

`database.cpp` → `main.cpp`

is added, DFS successfully identifies the circular dependency.

9. Conclusion

CodeClean successfully demonstrates eight fundamental Data Structures and Algorithms through a practical code management application.

- HashMap provides $O(1)$ variable lookup for fast searches.
- Stack enables efficient Undo/Redo operations using LIFO ordering.
- Queue ensures fair file inspection through FIFO processing.
- HashSet and HashMap allow instant keyword and function validation.
- Merge Sort guarantees stable $O(n \log n)$ ranking of functions.
- Graphs represented through Adjacency Lists efficiently model file dependencies.
- BFS finds the shortest bug-trace path in dependency graphs.
- Min-Heap always retrieves the lowest-cost optimization option.

Each data structure was selected because it provides the most efficient solution for its specific problem. The application executes correctly using preloaded sample data, handles invalid inputs and empty structures gracefully, and displays complexity information during execution to reinforce DSA concepts.

The project demonstrates how fundamental Data Structures and Algorithms are used in real-world software tools such as:

- SonarQube for code quality and static analysis
- IntelliJ IDEA for dependency navigation and code intelligence
- Visual Studio Code for undo/redo and developer productivity features

Thus, CodeClean effectively bridges theoretical DSA concepts with practical software engineering applications.

10. GitHub Repository Link

Project Source Code Repository:

GitHub: <https://github.com/sahil24302021/DSA-FINAL-CASE-STUDY>

